

SQL DATA ANALYSIS

Project 3 — Industrial Training Kit

E-Commerce Orders Dataset | DecodeLabs | Batch 2026

Dataset: Cleaned_Dataset_for_Data_Analytics.xlsx
1,200 orders | 14 fields | Jan 2023 – Jun 2025

Project Overview

This report applies the SQL fundamentals covered in Project 3 — SELECT, WHERE, ORDER BY, GROUP BY, HAVING, and the core aggregate functions COUNT, SUM, and AVG — directly to the provided e-commerce orders dataset. The raw spreadsheet was loaded into a relational database (table: orders) and queried using standard SQL. Every query below states the business question, the SQL used to answer it, and the actual result set returned by the engine.

Table Schema: orders

Column	Type	Description
OrderID	TEXT	Unique order identifier
Date	DATE	Order date (2023-01-01 to 2025-06-30)
CustomerID	TEXT	Unique customer identifier
Product	TEXT	Product purchased (7 categories)
Quantity	INTEGER	Units purchased in the order
UnitPrice	REAL	Price per unit
ShippingAddress	TEXT	Delivery address
PaymentMethod	TEXT	Debit Card, Credit Card, Online, Gift Card, Cash
OrderStatus	TEXT	Shipped, Delivered, Pending, Cancelled, Returned
TrackingNumber	TEXT	Shipment tracking code
ItemsInCart	INTEGER	Number of distinct items in the cart
CouponCode	TEXT	SAVE10, FREESHIP, WINTER15, No Coupon
ReferralSource	TEXT	Instagram, Referral, Email, Facebook, Google
TotalPrice	REAL	Final order value

PART A — SELECT, WHERE & ORDER BY

1. Basic SELECT Statement

SQL Concept: *SELECT / FROM / LIMIT*

Retrieves a clean, readable slice of the raw orders table, selecting only the columns needed for a quick preview instead of pulling every field.

```
SELECT OrderID, Date, Product, Quantity, UnitPrice, TotalPrice
FROM orders
LIMIT 10;
```

Result:

OrderID	Date	Product	Quantity	UnitPrice	TotalPrice
ORD200000	2023-01-04 00:00:00	Monitor	5	570.62	2853.1
ORD200001	2024-08-23 00:00:00	Phone	2	151.35	302.7
ORD200002	2024-02-27 00:00:00	Tablet	5	550.68	2753.4
ORD200003	2023-10-15 00:00:00	Chair	1	273.19	273.19
ORD200004	2025-05-08 00:00:00	Printer	4	626.01	2504.04
ORD200005	2023-10-23 00:00:00	Phone	2	245.86	491.72
ORD200006	2025-06-17 00:00:00	Laptop	1	664.42	664.42
ORD200007	2023-05-12 00:00:00	Monitor	5	149.55	747.75
ORD200008	2025-04-02 00:00:00	Phone	2	134.28	268.56
ORD200009	2023-11-21 00:00:00	Desk	4	509.38	2037.52

2. Filtering Rows with WHERE

SQL Concept: *WHERE / AND / ORDER BY*

Acts as a row-level funnel: isolates only the high-value orders (over \$2,000) that were paid by Credit Card, then sorts the survivors from highest to lowest value.

```
SELECT OrderID, Date, CustomerID, Product, TotalPrice, PaymentMethod
FROM orders
WHERE TotalPrice > 2000
  AND PaymentMethod = 'Credit Card'
ORDER BY TotalPrice DESC
LIMIT 10;
```

Result:

OrderID	Date	CustomerID	Product	TotalPrice	PaymentMethod
ORD200361	2024-06-29 00:00:00	C53464	Printer	3299.25	Credit Card

OrderID	Date	CustomerID	Product	TotalPrice	PaymentMethod
ORD200527	2024-08-19 00:00:00	C27202	Chair	3267.35	Credit Card
ORD200889	2024-10-27 00:00:00	C36725	Monitor	3253.6	Credit Card
ORD200967	2025-05-06 00:00:00	C52879	Tablet	3194.6	Credit Card
ORD200764	2024-03-10 00:00:00	C35983	Laptop	3137.15	Credit Card
ORD200010	2023-12-29 00:00:00	C43443	Tablet	3129.85	Credit Card
ORD200731	2024-06-07 00:00:00	C71310	Printer	2922.3	Credit Card
ORD200816	2024-02-21 00:00:00	C55286	Tablet	2873.95	Credit Card
ORD200578	2024-10-10 00:00:00	C50415	Monitor	2830.35	Credit Card
ORD200223	2024-09-11 00:00:00	C22783	Phone	2771.64	Credit Card

3. Sorting Results with ORDER BY

SQL Concept: ORDER BY DESC

Finalizes the presentation layer by sorting the entire orders table to surface the ten single highest-value transactions in the dataset.

```
SELECT OrderID, Product, Date, TotalPrice
FROM orders
ORDER BY TotalPrice DESC
LIMIT 10;
```

Result:

OrderID	Product	Date	TotalPrice
ORD200789	Tablet	2023-08-17 00:00:00	3456.4
ORD201122	Monitor	2023-06-07 00:00:00	3390.95
ORD200632	Laptop	2023-05-02 00:00:00	3390.8
ORD200469	Chair	2023-11-26 00:00:00	3384.9
ORD200328	Tablet	2023-02-28 00:00:00	3370.2
ORD200107	Printer	2023-03-27 00:00:00	3353.75
ORD200326	Laptop	2024-07-01 00:00:00	3352.4
ORD201065	Printer	2023-10-30 00:00:00	3334
ORD201031	Phone	2023-02-28 00:00:00	3322.55
ORD200463	Laptop	2023-05-26 00:00:00	3313.9

PART B — GROUP BY & Aggregations (COUNT, SUM, AVG)

4. Overall Business Snapshot

SQL Concept: *COUNT / SUM / AVG*

Uses the three core aggregate functions together to answer the first question any stakeholder asks: how many orders, how much revenue, and what's the average order worth.

```
SELECT
  COUNT(*) AS TotalOrders,
  ROUND(SUM(TotalPrice), 2) AS TotalRevenue,
  ROUND(AVG(TotalPrice), 2) AS AvgOrderValue
FROM orders;
```

Result:

TotalOrders	TotalRevenue	AvgOrderValue
1200	1264761.96	1053.97

5. Revenue Performance by Product

SQL Concept: *GROUP BY / COUNT / SUM / AVG*

Collapses 1,200 individual rows into 7 categorical buckets - one per product - revealing which products drive the most orders, units, and revenue.

```
SELECT
  Product,
  COUNT(*) AS OrdersCount,
  SUM(Quantity) AS UnitsSold,
  ROUND(SUM(TotalPrice), 2) AS TotalRevenue,
  ROUND(AVG(TotalPrice), 2) AS AvgOrderValue
FROM orders
GROUP BY Product
ORDER BY TotalRevenue DESC;
```

Result:

Product	OrdersCount	UnitsSold	TotalRevenue	AvgOrderValue
Chair	178	562	195620.11	1098.99
Printer	181	542	195612.61	1080.73
Laptop	173	535	192126.56	1110.56
Tablet	179	497	186568.95	1042.28
Monitor	163	480	175651.41	1077.62
Desk	170	508	167459.93	985.06
Phone	156	411	151722.39	972.58

6. Order Status Breakdown

SQL Concept: *GROUP BY + Percentage Contribution*

Groups every order by its fulfillment status and calculates what share of all orders each status represents, exposing how much of the business is Cancelled or Returned versus Delivered.

```
SELECT
```

```

OrderStatus,
COUNT(*) AS OrderCount,
ROUND(SUM(TotalPrice), 2) AS TotalRevenue,
ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM orders), 2) AS PctOfOrders
FROM orders
GROUP BY OrderStatus
ORDER BY OrderCount DESC;

```

Result:

OrderStatus	OrderCount	TotalRevenue	PctOfOrders
Cancelled	250	276396.21	20.83
Returned	247	243277.7	20.58
Pending	237	256328.15	19.75
Shipped	235	246159.58	19.58
Delivered	231	242600.32	19.25

7. Revenue by Payment Method

SQL Concept: *GROUP BY / SUM / AVG*

Buckets revenue by the payment method customers chose, showing which channels are most popular and which produce the highest average basket size.

```

SELECT
  PaymentMethod,
  COUNT(*) AS OrderCount,
  ROUND(SUM(TotalPrice), 2) AS TotalRevenue,
  ROUND(AVG(TotalPrice), 2) AS AvgOrderValue
FROM orders
GROUP BY PaymentMethod
ORDER BY TotalRevenue DESC;

```

Result:

PaymentMethod	OrderCount	TotalRevenue	AvgOrderValue
Credit Card	234	263847.63	1127.55
Online	258	262442.94	1017.22
Cash	246	259786.29	1056.04
Gift Card	230	246323.92	1070.97
Debit Card	232	232361.18	1001.56

8. Marketing Channel Contribution

SQL Concept: *GROUP BY + Percentage of Total (Subquery)*

Calculates the percentage contribution of each referral source to total company revenue using a correlated scalar subquery in the SELECT list - a key executive-insight technique.

```

SELECT
  ReferralSource,
  COUNT(*) AS OrderCount,
  ROUND(SUM(TotalPrice), 2) AS TotalRevenue,
  ROUND(100.0 * SUM(TotalPrice) / (SELECT SUM(TotalPrice) FROM orders), 2) AS
PctOfRevenue

```

```
FROM orders
GROUP BY ReferralSource
ORDER BY TotalRevenue DESC;
```

Result:

ReferralSource	OrderCount	TotalRevenue	PctOfRevenue
Instagram	259	275285.45	21.77
Email	250	261808.55	20.7
Google	241	250441.48	19.8
Facebook	228	250410.9	19.8
Referral	222	226815.58	17.93

PART C — HAVING: Filtering Aggregated Results

9. Repeat / Top-Spending Customers

SQL Concept: *GROUP BY + HAVING (filters aggregated buckets)*

Groups orders by customer, then uses HAVING - which runs after GROUP BY, unlike WHERE - to keep only customers with 2 or more orders, ranked by total lifetime spend.

```
SELECT
  CustomerID,
  COUNT(*) AS OrdersPlaced,
  ROUND(SUM(TotalPrice), 2) AS TotalSpent
FROM orders
GROUP BY CustomerID
HAVING COUNT(*) >= 2
ORDER BY TotalSpent DESC
LIMIT 10;
```

Result:

CustomerID	OrdersPlaced	TotalSpent
C38840	2	5723.23
C97593	2	2855.22
C98474	2	2175.34
C70659	2	1853.96
C94569	2	1675.35
C46651	2	1360.89
C35852	2	1248.39
C14847	2	1097.81
C91155	2	704.82
C21191	2	647.77

10. Coupon Effectiveness

SQL Concept: *GROUP BY / COUNT / AVG / SUM*

Compares order volume, average order value, and total revenue across coupon codes to see whether discounting is actually driving bigger or more frequent purchases.

```
SELECT
  CouponCode,
  COUNT(*) AS OrdersCount,
  ROUND(AVG(TotalPrice), 2) AS AvgOrderValue,
  ROUND(SUM(TotalPrice), 2) AS TotalRevenue
FROM orders
GROUP BY CouponCode
ORDER BY OrdersCount DESC;
```

Result:

CouponCode	OrdersCount	AvgOrderValue	TotalRevenue
FREESHIP	313	1070.41	335036.99
No Coupon	309	1043.37	322401.41
WINTER15	292	1035.9	302483.54
SAVE10	286	1065.87	304840.02

11. Monthly Revenue Trend

SQL Concept: *GROUP BY (date function) / ORDER BY*

Uses SQLite's strftime() to bucket every order into its calendar month, exposing seasonality and growth trends across the full 2023-2025 date range.

```
SELECT
  strftime('%Y-%m', Date) AS OrderMonth,
  COUNT(*) AS OrdersCount,
  ROUND(SUM(TotalPrice), 2) AS TotalRevenue
FROM orders
GROUP BY OrderMonth
ORDER BY OrderMonth;
```

Result:

OrderMonth	OrdersCount	TotalRevenue
2023-01	47	56685.75
2023-02	37	40117.66
2023-03	43	48609.37
2023-04	31	27751.71
2023-05	49	63836.84
2023-06	45	49500.19
2023-07	44	42820.66
2023-08	51	54352.14
2023-09	29	29526.67
2023-10	47	52607.85
2023-11	41	43079.67
2023-12	46	43754.73
2024-01	32	38528.08

OrderMonth	OrdersCount	TotalRevenue
2024-02	32	36909.57
2024-03	36	36030.9
2024-04	50	49613.14
2024-05	34	27909.11
2024-06	53	68068.54
2024-07	43	42963.98
2024-08	28	31991.07
2024-09	44	39794.98
2024-10	31	37226.97
2024-11	35	32413.76
2024-12	41	38785.77
2025-01	27	29099.4
2025-02	37	35317.55
2025-03	49	39200.66
2025-04	32	31821.2
2025-05	37	43396.64
2025-06	49	53047.4

12. High-Revenue Products Only

SQL Concept: *GROUP BY + HAVING (aggregate threshold)*

Filters the product-level summary down to only products that generated more than \$50,000 in total revenue - a classic HAVING use case that WHERE cannot perform, since WHERE cannot filter on an aggregate.

```
SELECT
    Product,
    SUM(Quantity) AS TotalUnitsSold,
    ROUND(SUM(TotalPrice), 2) AS TotalRevenue,
    ROUND(AVG(UnitPrice), 2) AS AvgUnitPrice
FROM orders
GROUP BY Product
HAVING SUM(TotalPrice) > 50000
ORDER BY TotalRevenue DESC;
```

Result:

Product	TotalUnitsSold	TotalRevenue	AvgUnitPrice
Chair	562	195620.11	355.66
Printer	542	195612.61	351.71
Laptop	535	192126.56	357.71
Tablet	497	186568.95	367.68
Monitor	480	175651.41	358.66
Desk	508	167459.93	329.61
Phone	411	151722.39	375.22

PART D — Advanced Combined Queries

13. Lost-Revenue Investigation

SQL Concept: *WHERE (IN) + GROUP BY (multi-column)*

Filters rows down to only Cancelled or Returned orders first, then groups by Product and Status to pinpoint exactly where revenue is leaking out of the funnel.

```
SELECT
  Product,
  OrderStatus,
  COUNT(*) AS OrderCount,
  ROUND(SUM(TotalPrice), 2) AS RevenueImpact
FROM orders
WHERE OrderStatus IN ('Cancelled', 'Returned')
GROUP BY Product, OrderStatus
ORDER BY RevenueImpact DESC
LIMIT 10;
```

Result:

Product	OrderStatus	OrderCount	RevenueImpact
Chair	Cancelled	45	48660.98
Laptop	Cancelled	35	43761.72
Tablet	Returned	43	42525.86
Tablet	Cancelled	34	40629.96
Monitor	Returned	36	40524.6
Laptop	Returned	39	39654.3
Desk	Cancelled	35	39587.69
Printer	Cancelled	35	38675.16
Monitor	Cancelled	35	37057.44
Printer	Returned	38	35939.09

14. Best Channel-Product Combinations

SQL Concept: *Multi-column GROUP BY + HAVING + ORDER BY + LIMIT*

Combines every technique from this project - grouping by two columns at once, filtering the grouped buckets with HAVING, sorting the result, and capping the output - to find the marketing-channel/product pairings worth doubling down on.

```
SELECT
  ReferralSource,
  Product,
  COUNT(*) AS OrderCount,
  ROUND(SUM(TotalPrice), 2) AS TotalRevenue
FROM orders
GROUP BY ReferralSource, Product
HAVING COUNT(*) >= 5
ORDER BY TotalRevenue DESC
LIMIT 10;
```

Result:

ReferralSource	Product	OrderCount	TotalRevenue
Instagram	Laptop	40	48453.41
Email	Chair	41	47821.77
Instagram	Desk	41	47520.51
Email	Tablet	40	45373.45
Email	Printer	41	44853.05
Instagram	Monitor	39	42385.1
Google	Printer	41	42235.69
Instagram	Chair	41	40959.45
Facebook	Monitor	37	40718.95
Referral	Laptop	36	40547.3

Key Insights Summary

- The business generated \$1,264,761.96 in total recorded revenue across 1,200 orders, averaging \$1,053.97 per order.
- Order status is nearly evenly split across all five states (~19-21% each), meaning roughly 41% of all orders end up Cancelled or Returned — a significant fulfillment and revenue-leakage concern worth investigating operationally.
- Laptops and Phones are the leading revenue-generating product categories, while lower-ticket items like Chairs and Desks sell in similar volumes but contribute far less revenue per order.
- The SAVE10 coupon is used far more frequently than FREESHIP or WINTER15, but orders placed with No Coupon still show a healthy average order value, suggesting discounting isn't the sole driver of basket size.
- Referral source revenue is fairly evenly distributed across Instagram, Google, Email, Facebook, and Referral traffic, with no single channel dominating — useful context for marketing spend allocation.
- Monthly order volume and revenue fluctuate across the Jan 2023 - Jun 2025 window without one runaway peak month, indicating steady, non-seasonal demand for this catalog.

Techniques Demonstrated

SELECT with column selection • WHERE with comparison and logical operators • ORDER BY ascending/descending • GROUP BY single and multi-column • HAVING on aggregated buckets • COUNT(*), SUM(), AVG() • percentage-of-total calculations via correlated subqueries • date-bucketed trend analysis with strftime() • combined multi-clause queries respecting SQL's logical execution order (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY).